

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	B.VANI	Reg. No.:	192472348
Section / Batch:	CSE-AI/2024	Date:	

Code of Conduct

I B.VANI(192472348) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: B.VANI

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.

Apply your algorithm to:

User: "I want to book a flight."

Agent: "Where would you like to travel?"

User: "I want to go to Delhi."

Agent: "From which city?"

User: "From Chennai."

Your algorithm should maintain the following slots:

Departure City

Destination City

Travel Date

Number of Passengers

Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach

Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

DSA0306-NATURAL LANGUAGE PROCESSING

CO5 AT 2

B.VANI(192472348)

Q1. REFERENCE RESOLUTION USING MULTIPLE CONSTRAINTS

a) Entities and Referring Expressions

Given discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Entities:

1. Meena
2. Kavya
3. Laptop
4. Project

Referring expressions:

- she → Meena or Kavya
- one → laptop
- her → Meena or Kavya
- She → Meena or Kavya
- her → Meena or Kavya
- it → laptop

b) Possible Antecedents

Referring Expression	Possible Antecedents
she	Meena, Kavya
one	laptop
her	Meena, Kavya
She	Meena, Kavya
her	Meena, Kavya
it	laptop

c) Constraint Analysis

Gender and Number:

Both Meena and Kavya are singular females, so both satisfy gender and number agreement for "she" and "her".

Recency:

For the first "she", Kavya is the closest candidate because she appears immediately before the pronoun.

Grammatical Role:

"Meena gave Kavya a laptop" makes Meena the giver and Kavya the recipient. "She needed one for her project" can naturally refer to Kavya because the recipient needs the laptop.

Semantic Compatibility:

The pronoun "she" should refer to a person who can need a laptop. "one" refers to the laptop because it is the object being needed.

Discourse Coherence:

After Meena gives the laptop to Kavya, it is coherent for Kavya to need the laptop, thank Meena, and start using it.

d) Final Resolution

Meena → her

Kavya → she → She → her

laptop → one → it

Therefore:

she = Kavya

one = laptop

She = Kavya

her = Meena in "She thanked her"

it = laptop

e) Resolved Discourse

"Meena gave Kavya a laptop because Kavya needed the laptop for her project. Kavya thanked Meena and started using the laptop."

f) Avoiding Incorrect Resolution

When multiple antecedents have similar grammatical properties, the algorithm should combine several constraints instead of relying only on gender or recency.

A scoring method can be used:

Score = Gender + Number + Recency + Grammatical Role + Semantic Compatibility

+ Discourse Coherence

The candidate with the highest score is selected. Semantic compatibility and grammatical-role information prevent incorrect resolutions when both candidates are grammatically valid.

Pseudocode

START

Input discourse

Identify entities and referring expressions

FOR each referring expression

 Generate possible antecedents

 FOR each candidate

 Check gender agreement

 Check number agreement

 Calculate recency score

 Check grammatical role

 Check semantic compatibility

 Check discourse coherence

 Calculate total score

 END FOR

 Select candidate with highest score

END FOR

Replace referring expressions with selected antecedents

Display resolved discourse

STOP

Result

The algorithm resolves the discourse by identifying Kavya as the antecedent of the pronouns referring to the person who needed the laptop, while Meena becomes the

antecedent of "her" in "She thanked her". The laptop is the antecedent of "one" and "it".

Q2. ALGORITHM FOR ENTITY CHAINS AND DISCOURSE COHERENCE

a) Entities and Referring Expressions

Text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Entities:

1. Anjali
2. Laptop
3. Processor
4. Python
5. Computer
6. Machine learning application

Referring expressions:

- The laptop → laptop
- She → Anjali
- it → laptop
- Anjali → Anjali
- the computer → laptop

b) Entity Linking

Anjali → She → Anjali

laptop → The laptop → it → the computer

processor → processor

Python → Python

machine learning application → application

c) Entity Chains

Entity Chain 1:

Anjali → She → Anjali

Entity Chain 2:

laptop → The laptop → it → the computer

Entity Chain 3:

processor

Entity Chain 4:

Python

Entity Chain 5:

machine learning application

d) Entity Continuity

Sentence 1:

Anjali and laptop introduced.

Sentence 2:

Laptop is continued.

Sentence 3:

Anjali and laptop are continued using "She" and "it".

Sentence 4:

Anjali and laptop are continued using "Anjali" and "the computer".

Thus, the main entities remain active across multiple sentences.

A simple continuity measure can be estimated as:

Entity Continuity = Number of linked entity mentions / Total entity mentions

The major entities have strong continuity throughout the discourse.

e) Coherence

The discourse is coherent because each sentence maintains links to previously introduced entities.

The sequence is:

Anjali buys laptop

↓

Laptop description

↓

Anjali uses laptop

↓

Anjali uses computer/laptop for application development

f) Effect of Breaking an Entity Chain

If "She" were incorrectly linked to another person, or "the computer" were interpreted as a new unrelated device, the connection between sentences would weaken. The reader or NLP system may believe that a different person or object is involved, reducing discourse coherence.

Pseudocode

START

Input discourse

Split discourse into sentences

Extract entities and referring expressions

FOR each referring expression

 Find candidate entities

 Compare lexical similarity

 Check semantic compatibility

 Check recency and grammatical role

 Link expression to best entity

END FOR

Construct entity chains

Calculate entity continuity

IF major entities remain connected across sentences

 Coherence = HIGH

ELSE

 Coherence = LOW

END IF

Display entity chains and coherence result

STOP

Result

The discourse is coherent because the entity chains for Anjali and the laptop continue across the sentences. Maintaining these chains allows the system to preserve meaning and continuity.

Q3. ALGORITHM FOR IDENTIFYING DISCOURSE RELATIONS

a) Discourse Units

The discourse contains four units:

D1: The server crashed unexpectedly.

D2: As a result, users could not access the application.

D3: The technical team restarted the server.

D4: After that, the system became available again.

b) Explicit Discourse Markers

D2 → "As a result"

D4 → "After that"

c) Semantic Relations

D1 → D2:

Cause–Effect

The server crash caused users to lose access.

D2 → D3:

Problem–Solution

The access problem led to the technical team restarting the server.

D3 → D4:

Sequence

After the restart, the system became available.

d) Discourse Structure

D1: Server crashed

↓ Cause

D2: Users could not access application

↓ Problem

D3: Technical team restarted server

↓ Sequence/Resolution

D4: System became available

e) Pseudocode

START

Input discourse

Split into discourse units

FOR each consecutive pair of units

 Identify explicit discourse markers

 Analyze semantic meaning

 IF marker = "as a result"

 Relation = Cause-Effect

 ELSE IF marker = "after that"

 Relation = Sequence

 ELSE IF second unit resolves first unit

 Relation = Problem-Solution

 ELSE IF second unit adds information

 Relation = Elaboration

 ELSE

 Relation = Context-Based Relation

 END IF

END FOR

Construct discourse structure

Display relations

STOP

f) Justification

The identified relations make the discourse logically coherent:

Crash → Access Problem → Restart → Recovery

The first sentence introduces a problem, the second gives its effect, the third describes the solution, and the fourth describes the result of the solution.

Result

The discourse contains Cause–Effect, Problem–Solution, and Sequence relations. These relations establish a clear logical progression from server failure to system recovery.

Q4. ALGORITHM FOR DIALOGUE ACT CLASSIFICATION

a) Dialogue Act Classification

Conversation:

User: "Hello, I need help with my assignment."

Dialogue Act = Greeting + Request

Agent: "Sure, what problem are you facing?"

Dialogue Act = Question

User: "I do not understand machine learning."

Dialogue Act = Inform

Agent: "I can explain the basic concepts to you."

Dialogue Act = Offer

b) Linguistic Features

The algorithm can examine:

Greeting words → hello, hi

Question words/patterns → what, why, how, ?

Request patterns → need help, please, can you

Informative statements → I do not understand...

Offer patterns → I can, I will, let me

Confirmation patterns → yes, okay, correct

c) Communicative Intentions

"Hello..." → Greeting

"I need help..." → Request

"What problem..." → Question

"I do not understand..." → Inform

"I can explain..." → Offer

d) Dialogue-Act Sequence

Greeting + Request → Question → Inform → Offer

e) Pseudocode

START

Input conversation

Split into utterances

FOR each utterance

 Convert to lowercase

 Tokenize utterance

 Extract keywords and linguistic patterns

 IF greeting pattern found

 Act = Greeting

 ELSE IF request pattern found

 Act = Request

 ELSE IF question pattern found

 Act = Question

 ELSE IF offer pattern found

 Act = Offer

 ELSE IF confirmation pattern found

 Act = Confirmation

 ELSE

 Act = Inform

 END IF

 Store dialogue act

END FOR

Display dialogue-act sequence

STOP

f) Importance

Dialogue-act recognition helps the conversational agent understand what the user is trying to accomplish. For example, a Request requires assistance, a Question requires an answer, and an Offer can be accepted or rejected. Therefore, the recognized

dialogue act guides the agent toward an appropriate next response.

Result

The conversation is classified as:

Greeting + Request → Question → Inform → Offer

Dialogue-act recognition allows the agent to select responses according to the user's communicative intention.

Q5. ALGORITHM FOR DIALOGUE STATE TRACKING

a) Initial Dialogue State

Departure City = Unknown

Destination City = Unknown

Travel Date = Unknown

Number of Passengers = Unknown

b) State Updates

User:

"I want to book a flight."

Detected information:

Intent = Book Flight

State:

Departure City = Unknown

Destination City = Unknown

Travel Date = Unknown

Number of Passengers = Unknown

Agent:

"Where would you like to travel?"

User:

"I want to go to Delhi."

Detected:

Destination City = Delhi

State:

Departure City = Unknown

Destination City = Delhi

Travel Date = Unknown

Number of Passengers = Unknown

Agent:

"From which city?"

User:

"From Chennai."

Detected:

Departure City = Chennai

Final state:

Departure City = Chennai

Destination City = Delhi

Travel Date = Unknown

Number of Passengers = Unknown

c) Missing Information

The missing slots are:

Travel Date

Number of Passengers

Therefore, the next appropriate question is:

"What is your travel date?"

After receiving the date, the agent should ask:

"How many passengers are travelling?"

d) Pseudocode

START

Initialize dialogue state

Departure = NULL

Destination = NULL

TravelDate = NULL

Passengers = NULL

FOR each user utterance

 Extract entities and slot values

 IF departure city detected

 Update Departure

 END IF

 IF destination city detected

 Update Destination

 END IF

 IF travel date detected

 Update TravelDate

 END IF

 IF passenger number detected

 Update Passengers

 END IF

Identify missing slots

IF Destination is NULL

 Ask destination

ELSE IF Departure is NULL

 Ask departure city

ELSE IF TravelDate is NULL

 Ask travel date

ELSE IF Passengers is NULL

```

        Ask number of passengers
    ELSE
        Confirm booking details
    END IF
END FOR
STOP

```

e) Dialogue State Table

Turn	User Input	Departure	Destination	Travel Date	Passengers
1	I want to book a flight	Unknown	Unknown	Unknown	Unknown
2	I want to go to Delhi	Unknown	Delhi	Unknown	Unknown
3	From Chennai	Chennai	Delhi	Unknown	Unknown

f) Importance of Dialogue State Tracking

Dialogue state tracking maintains information collected over multiple turns. Without it, the agent may repeatedly ask for information that the user has already provided. For example, after receiving "Delhi" and "Chennai", the system remembers:

Chennai → Delhi

and asks only for the remaining information. This improves coherence, avoids repetition, and allows the conversation to progress logically.

Result

The dialogue state tracking algorithm successfully maintains the user's booking information across multiple turns. The current state is Chennai as the departure city and Delhi as the destination, with travel date and number of passengers still missing.